

Source Code

Story Co-Writer Agent · Capgemini Excellencer AgentifAI Buildathon (Round 2)

Platform: n8n · Agents: gpt-4.1-mini · PDF: pure-JavaScript generator

1. Architecture

Single end-to-end pipeline. One request runs three LLM agents in sequence, then renders a PDF inside the workflow:

```
Webhook / Manual Trigger
-> Normalize Writer Input
-> Scene Planner Agent      (gpt-4.1-mini, temp 0.8, + session memory) -> 3 ranked scenes
-> Continuity Guard Agent  (gpt-4.1-mini, temp 0.1)                 -> validation + NCS
-> Story Bible Agent       (gpt-4.1-mini, temp 0.3)                 -> canonical bible
-> Assemble Final Report   (combine all outputs)
-> Generate PDF            (pure-JS PDF writer, zero dependencies)
-> Respond with Story Package (application/pdf)
```

2. Agent System Prompts

Scene Planner

You are NarrativeAI Scene Planner – a senior story co-writer AI. Your job is to generate exactly 3 ranked scene suggestions based on the story context provided.

For each scene use this format:

```
**Scene [N]: [Title]**
- Summary: (2-3 sentences describing what happens)
- Narrative Purpose: (structural role in the story arc)
- Character Impact: (how each key character is affected)
- Story Threads: (which unresolved threads are opened or closed)
- Opening Line: (a compelling first sentence for this scene)
```

Rank Scene 1 as the strongest narrative recommendation. Respect all writer constraints.

Continuity Guard

You are NarrativeAI Continuity Guard – precision narrative quality assurance. Validate each scene across 5 dimensions:

1. CHARACTER CONSISTENCY – Do behaviors match established personality and motivations?
2. TIMELINE INTEGRITY – Any temporal impossibilities or sequence errors?
3. LOCATION VALIDITY – Are positions and travel times plausible?
4. RELATIONSHIP DYNAMICS – Are established relationships respected?
5. CONSTRAINT COMPLIANCE – Do scenes honour the writer constraints?

For each scene output:

```
**[Scene Title] – Continuity Report**
- Status: PASS / WARNING / FAIL
- Issues: (each with confidence % and reference)
- Fixes: (specific corrections)
- Scene NCS: [0-100]
```

```
**Overall Narrative Coherence Score (NCS): [0-100]**
```

```
**Recommendation: APPROVE / REVISE / REJECT**
```

Story Bible Agent

You are NarrativeAI Story Librarian – keeper of the canonical Story Bible. Extract and merge narrative facts from the new content into the Story Bible. Return the COMPLETE updated Story Bible in this exact structure:

```
## CHARACTERS
[Each: name, age, traits, arc, relationships]

## WORLD STATE
[Locations, organizations, world rules]

## EVENT HISTORY
[Chronological key events]

## LORE
[Canon facts, mythology, world rules]

## LOCKED ELEMENTS
[Canonical events that cannot be changed]

## CHANGE LOG
[Summary of what was added or modified in this update]
```

3. src/generate-pdf.js

Pure-JavaScript PDF generator that runs inside the n8n Code node (no npm imports available on n8n Cloud).

```
const data = $input.first().json;

function buildNarrativePdf(data) {
  var NL = String.fromCharCode(10);
  var BS = String.fromCharCode(92);
  var SCENES = data.scene_suggestions || '';
  var CONT = data.continuity_report || '';
  var BIBLE = data.updated_story_bible || '';
  var SESSION = data.session_id || '';
  var pageW = 595.28, pageH = 841.89, margin = 50;
  var topY = pageH - margin, bottomY = margin + 6;

  function ascii(s) {
    s = String(s);
    s = s.split(String.fromCharCode(0x2014)).join('-').split(String.fromCharCode(0x2013)).join('-');
    s = s.split(String.fromCharCode(0x2018)).join('').split(String.fromCharCode(0x2019)).join('');
    s = s.split(String.fromCharCode(0x201c)).join('').split(String.fromCharCode(0x201d)).join('');
    s = s.split(String.fromCharCode(0x2022)).join('-').split(String.fromCharCode(0x2026)).join('...');
    s = s.split(String.fromCharCode(0x00a0)).join(' ');
    var out = '';
    for (var i = 0; i < s.length; i++) { var c = s.charCodeAt(i); if (c >= 32 && c <= 126) out += s.charAt(i); }
    return out;
  }

  function esc(s) {
    s = ascii(s); var out = '';
    for (var i = 0; i < s.length; i++) { var ch = s.charAt(i); if (ch === '(' || ch === ')') || ch === BS) out += BS + ch; else out += ch; }
    return out;
  }

  var lines = [];
  function wrap(text, font, size, indent) {
    var usable = (pageW - 2 * margin) - (indent || 0);
    var maxChars = Math.max(10, Math.floor(usable / (size * 0.52)));
    var words = ascii(text).split(' ');
    var cur = '';
    for (var i = 0; i < words.length; i++) {
      var w = words[i]; if (!w) continue;
      if (!cur) cur = w;
      else if ((cur + ' ' + w).length <= maxChars) cur += ' ' + w;
      else { lines.push({ text: cur, font: font, size: size, indent: indent || 0 }); cur = w; }
    }
    if (cur) lines.push({ text: cur, font: font, size: size, indent: indent || 0 });
  }

  function gap(h) { lines.push({ gap: h }); }
  function strip(s) { return s.split('***').join(''); }
  function renderMarkdown(block) {
    var arr = String(block).split(NL);
    for (var i = 0; i < arr.length; i++) {
      var line = arr[i].replace(/[/]\+$/g, '');
      if (!line) { gap(5); continue; }
      if (line.indexOf('## ') === 0) { gap(4); wrap(strip(line.substring(3)), 'F2', 13, 0); gap(2); continue; }
      if (line.indexOf('***') === 0 && line.lastIndexOf('***') === line.length - 2 && line.indexOf('- ') === -1) {
        gap(3); wrap(strip(line), 'F2', 11, 0); continue; }
      if (line.indexOf('***') === 0) { gap(3); wrap(strip(line), 'F2', 10.5, 0); continue; }
      if (line.indexOf('- ') === 0) { wrap('- ' + strip(line.substring(2)), 'F1', 9.5, 12); continue; }
      wrap(strip(line), 'F1', 9.5, 0);
    }
  }

  lines.push({ text: 'NarrativeAI', font: 'F2', size: 11, indent: 0 });
  lines.push({ text: 'Story Production Report', font: 'F2', size: 20, indent: 0 });
  gap(3);
  lines.push({ text: 'Pipeline: Scene Planner > Continuity Guard > Story Bible', font: 'F1', size: 9, indent: 0 });
  if (SESSION) lines.push({ text: 'Session: ' + SESSION, font: 'F1', size: 9, indent: 0 });
  gap(10);
  lines.push({ text: '1. Scene Suggestions', font: 'F2', size: 13, indent: 0 }); gap(4);
  renderMarkdown(SCENES); gap(8);
  lines.push({ text: '2. Continuity Validation', font: 'F2', size: 13, indent: 0 }); gap(4);
  renderMarkdown(CONT); gap(8);
  lines.push({ text: '3. Updated Story Bible', font: 'F2', size: 13, indent: 0 }); gap(4);
  renderMarkdown(BIBLE);
  var pages = [], ops = [], y = topY;
  function flush() { pages.push(ops.join(NL)); ops = []; y = topY; }
  for (var i = 0; i < lines.length; i++) {
    var ln = lines[i];
    if (ln.gap !== undefined) { y -= ln.gap; if (y < bottomY) flush(); continue; }
    var leading = ln.size * 1.45;
    if (y - leading < bottomY) flush();
    var x = (margin + (ln.indent || 0)).toFixed(2);
    var baseline = (y - ln.size).toFixed(2);
    ops.push('BT /' + ln.font + ' ' + ln.size + ' Tf 1 0 0 1 ' + x + ' ' + baseline + ' Tm (' + esc(ln.text) + ') Tj ET');
    y -= leading;
  }
  if (ops.length) flush();
}
```

```

if (pages.length === 0) pages.push('');
var nPages = pages.length;
var catalog = 1, pagesObj = 2, fReg = 3, fBold = 4, contentStart = 5, pageStart = 5 + nPages;
var kids = [];
for (var i = 0; i < nPages; i++) kids.push((pageStart + i) + ' 0 R');
var objects = [];
objects[catalog] = '<< /Type /Catalog /Pages ' + pagesObj + ' 0 R >>';
objects[pagesObj] = '<< /Type /Pages /Count ' + nPages + ' /Kids [' + kids.join(' ') + '] >>';
objects[fReg] = '<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica /Encoding /WinAnsiEncoding >>';
objects[fBold] = '<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica-Bold /Encoding /WinAnsiEncoding >>';
for (var i = 0; i < nPages; i++) {
  var stream = pages[i];
  var len = Buffer.byteLength(stream, 'latin1');
  objects[contentStart + i] = '<< /Length ' + len + ' >>' + NL + 'stream' + NL + stream + NL + 'endstream';
  objects[pageStart + i] = '<< /Type /Page /Parent ' + pagesObj + ' 0 R /MediaBox [0 0 ' + pageW.toFixed(2) + ' ' +
    pageH.toFixed(2) + '] /Resources << /Font << /F1 ' + fReg + ' 0 R /F2 ' + fBold + ' 0 R >> >> /Contents ' +
    (contentStart + i) + ' 0 R >>';
}
var total = 4 + 2 * nPages;
var header = '%PDF-1.4' + NL;
var body = '', offsets = [];
for (var n = 1; n <= total; n++) {
  offsets[n] = header.length + Buffer.byteLength(body, 'latin1');
  body += n + ' 0 obj' + NL + objects[n] + NL + 'endobj' + NL;
}
var xrefPos = header.length + Buffer.byteLength(body, 'latin1');
var xref = 'xref' + NL + '0 ' + (total + 1) + NL + '0000000000 65535 f ' + NL;
for (var n = 1; n <= total; n++) xref += String(offsets[n]).padStart(10, '0') + ' 00000 n ' + NL;
var trailer = 'trailer' + NL + '<< /Size ' + (total + 1) + ' /Root ' + catalog + ' 0 R >>' + NL + 'startxref' + NL +
  xrefPos + NL + '%%EOF';
return header + body + xref + trailer;
}

const fileName = 'NarrativeAI_Story_Production_Report.pdf';
const pdfStr = buildNarrativePdf(data);
const buffer = Buffer.from(pdfStr, 'latin1');
const binaryData = await this.helpers.prepareBinaryData(buffer, fileName, 'application/pdf');
return [{ json: { status: data.status || 'success', filename: fileName, message: data.message || 'PDF generated',
  session_id: data.session_id || '' }, binary: { data: binaryData } }];

```

4. workflow/narrativeai-workflow.json

Importable n8n workflow definition (15 nodes). Import via n8n → Workflows → Import from File.

```
{
  "name": "NarrativeAI - Story Co-Writer Pipeline",
  "nodes": [
    {
      "parameters": {
        "httpMethod": "POST",
        "path": "narrative-ai",
        "responseMode": "responseNode",
        "options": {}
      },
      "name": "Writer Input Webhook",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 2.1,
      "position": [
        0,
        704
      ],
      "webhookId": "3640b6f1-32c8-469f-9d92-f089e8932262"
    },
    {
      "parameters": {
        "assignments": {
          "assignments": [
            {
              "id": "ni-action",
              "name": "action",
              "value": "={{ $json.body?.action ?? $json.action ?? \"generate_scene\" }}",
              "type": "string"
            },
            {
              "id": "ni-content",
              "name": "content",
              "value": "={{ $json.body?.content ?? $json.content ?? \"\" }}",
              "type": "string"
            },
            {
              "id": "ni-genre",
              "name": "genre",
              "value": "={{ $json.body?.genre ?? $json.genre ?? \"general fiction\" }}",
              "type": "string"
            },
            {
              "id": "ni-tone",
              "name": "tone",
              "value": "={{ $json.body?.tone ?? $json.tone ?? \"neutral\" }}",
              "type": "string"
            },
            {
              "id": "ni-pacing",
              "name": "pacing",
              "value": "={{ $json.body?.pacing ?? $json.pacing ?? \"balanced\" }}",
              "type": "string"
            },
            {
              "id": "ni-constraints",
              "name": "constraints",
              "value": "={{ $json.body?.constraints ?? $json.constraints ?? \"none\" }}",
              "type": "string"
            },
            {
              "id": "ni-bible",
              "name": "storyBibleContext",
              "value": "={{ $json.body?.storyBibleContext ?? $json.storyBibleContext ?? \"No story bible provided.\" }}",
              "type": "string"
            },
            {
              "id": "ni-session",
              "name": "sessionId",
              "value": "={{ $json.body?.sessionId ?? $json.sessionId ?? $execution.id }}",
              "type": "string"
            }
          ]
        },
        "options": {}
      },
      "name": "Normalize Writer Input",
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [
        224,
        704
      ]
    }
  ],
  "options": {}
}
```

```

{
  "parameters": {
    "promptType": "define",
    "text": "STORY BIBLE:\n{{ $json.storyBibleContext }}\n\nGENRE: {{ $json.genre }} | TONE: {{ $json.tone }} | PACING: {{ $json.pacing }}\n\nWRITER CONSTRAINTS: {{ $json.constraints }}\n\nCURRENT NARRATIVE INPUT:\n{{ $json.content }}",
    "options": {
      "systemMessage": "You are NarrativeAI Scene Planner – a senior story co-writer AI. Your job is to generate exactly 3 ranked scene suggestions based on the story context provided.\n\nFor each scene use this format:\n**Scene [N]: [Title]**\n- Summary: (2-3 sentences describing what happens)\n- Narrative Purpose: (structural role in the story arc)\n- Character Impact: (how each key character is affected)\n- Story Threads: (which unresolved threads are opened or closed)\n- Opening Line: (a compelling first sentence for this scene)\n\nRank Scene 1 as the strongest narrative recommendation. Respect all writer constraints."
    }
  },
  "name": "Scene Planner Agent",
  "type": "@n8n/n8n-nodes-langchain.agent",
  "typeVersion": 3.1,
  "position": [
    672,
    0
  ]
},
{
  "parameters": {
    "model": {
      "__rl": true,
      "value": "gpt-4.1-mini",
      "mode": "list",
      "cachedResultName": "gpt-4.1-mini"
    },
    "builtInTools": {
      "webSearch": {
        "searchContextSize": "low"
      }
    },
    "options": {
      "temperature": 0.8
    }
  },
  "name": "Scene Planner LLM",
  "type": "@n8n/n8n-nodes-langchain.lmChatOpenAi",
  "typeVersion": 1.3,
  "position": [
    688,
    224
  ]
},
{
  "parameters": {
    "sessionIdType": "customKey",
    "sessionKey": "={{ $(\"Normalize Writer Input\").item.json.sessionId }}",
    "contextWindowLength": 10
  },
  "name": "Story Session Memory",
  "type": "@n8n/n8n-nodes-langchain.memoryBufferWindow",
  "typeVersion": 1.4,
  "position": [
    816,
    224
  ]
},
{
  "parameters": {
    "promptType": "define",
    "text": "CANONICAL STORY BIBLE:\n{{ $(\"Normalize Writer Input\").item.json.storyBibleContext }}\n\nWRITER CONSTRAINTS: {{ $(\"Normalize Writer Input\").item.json.constraints }}\n\nSCENES TO VALIDATE:\n{{ $json.output }}",
    "options": {
      "systemMessage": "You are NarrativeAI Continuity Guard – precision narrative quality assurance. Validate each scene across 5 dimensions:\n1. CHARACTER CONSISTENCY – Do behaviors match established personality and motivations?\n2. TIMELINE INTEGRITY – Any temporal impossibilities or sequence errors?\n3. LOCATION VALIDITY – Are positions and travel times plausible?\n4. RELATIONSHIP DYNAMICS – Are established relationships respected?\n5. CONSTRAINT COMPLIANCE – Do scenes honour the writer constraints?\n\nFor each scene output:\n**[Scene Title] – Continuity Report**\n- Status: PASS / WARNING / FAIL\n- Issues: (each with confidence % and reference)\n- Fixes: (specific corrections)\n- Scene NCS: [0-100]\n\n**Overall Narrative Coherence Score (NCS): [0-100]**\n\n**Recommendation: APPROVE / REVISE / REJECT**"
    }
  },
  "name": "Continuity Guard Agent",
  "type": "@n8n/n8n-nodes-langchain.agent",
  "typeVersion": 3.1,
  "position": [
    1024,
    0
  ]
},
{
  "parameters": {
    "model": {

```

```

    "__rl": true,
    "value": "gpt-4.1-mini",
    "mode": "list",
    "cachedResultName": "gpt-4.1-mini"
  },
  "builtInTools": {
    "webSearch": {
      "searchContextSize": "medium"
    }
  },
  "options": {
    "temperature": 0.1
  }
},
"name": "Continuity Guard LLM",
"type": "@n8n/n8n-nodes-langchain.lmChatOpenAi",
"typeVersion": 1.3,
"position": [
  1104,
  224
]
},
{
  "parameters": {
    "respondWith": "binary",
    "responseDataSource": "set",
    "inputFieldName": "data",
    "options": {
      "responseHeaders": {
        "entries": [
          {
            "name": "Content-Type",
            "value": "application/pdf"
          },
          {
            "name": "Content-Disposition",
            "value": "attachment; filename=\"NarrativeAI_Story_Production_Report.pdf\""
          }
        ]
      }
    }
  },
  "name": "Respond with Story Package",
  "type": "n8n-nodes-base.respondToWebhook",
  "typeVersion": 1.5,
  "position": [
    1504,
    800
  ]
},
{
  "parameters": {
    "promptType": "define",
    "text": "=EXISTING STORY BIBLE:\n{\n  $(\"Normalize Writer Input\").item.json.storyBibleContext\n}\n\nORIGINAL WRITER INPUT:\n{\n  $(\"Normalize Writer Input\").item.json.content\n}\n\nAPPROVED SCENE SUGGESTIONS:\n{\n  $(\"Scene Planner Agent\").item.json.output\n}\n\nCONTINUITY VALIDATION REPORT:\n{\n  $json.output\n}\n\nIntegrate the approved scenes that passed continuity validation into the canonical Story Bible.",
    "options": {
      "systemMessage": "You are NarrativeAI Story Librarian – keeper of the canonical Story Bible. Extract and merge narrative facts from the new content into the Story Bible. Return the COMPLETE updated Story Bible in this exact structure:\n\n## CHARACTERS\n[Each: name, age, traits, arc, relationships]\n\n## WORLD STATE\n[Locations, organizations, world rules]\n\n## EVENT HISTORY\n[Chronological key events]\n\n## LORE\n[Canon facts, mythology, world rules]\n\n## LOCKED ELEMENTS\n[Canonical events that cannot be changed]\n\n## CHANGE LOG\n[Summary of what was added or modified in this update]"
    }
  },
  "name": "Story Bible Agent",
  "type": "@n8n/n8n-nodes-langchain.agent",
  "typeVersion": 3.1,
  "position": [
    672,
    800
  ]
},
{
  "parameters": {
    "model": {
      "__rl": true,
      "value": "gpt-4.1-mini",
      "mode": "list",
      "cachedResultName": "gpt-4.1-mini"
    },
    "builtInTools": {
      "webSearch": {
        "searchContextSize": "low"
      }
    }
  },
  "options": {
    "temperature": 0.3
  }
}

```

```

    }
  },
  {
    "name": "Story Bible LLM",
    "type": "@n8n/n8n-nodes-langchain.lmChatOpenAi",
    "typeVersion": 1.3,
    "position": [
      752,
      1024
    ]
  }
],
{
  "parameters": {
    "content": "# NarrativeAI - Story Production Pipeline\n\nSingle end-to-end run (Manual Trigger or POST /webhook/narrative-ai):\n\n1. Scene Planner - generates 3 ranked scenes\n2. Continuity Guard - validates them (NCS)\n3. Story Bible Agent - canonizes approved scenes\n4. Assemble Final Report - combines everything into full_report\n\nInput fields: content, storyBibleContext, genre, tone, pacing, constraints, sessionId",
    "height": 200,
    "width": 300,
    "color": 3
  },
  {
    "name": "Sticky Note",
    "type": "n8n-nodes-base.stickyNote",
    "typeVersion": 1,
    "position": [
      -48,
      -48
    ]
  }
],
{
  "parameters": {},
  {
    "name": "Run Test (Click Execute)",
    "type": "n8n-nodes-base.manualTrigger",
    "typeVersion": 1,
    "position": [
      0,
      1456
    ]
  }
],
{
  "parameters": {
    "mode": "manual",
    "includeOtherFields": false,
    "assignments": {
      "assignments": [
        {
          "id": "st-action",
          "name": "action",
          "type": "string",
          "value": "generate_scene"
        },
        {
          "id": "st-content",
          "name": "content",
          "type": "string",
          "value": "A detective discovers their partner has been lying about the case for weeks."
        },
        {
          "id": "st-genre",
          "name": "genre",
          "type": "string",
          "value": "noir thriller"
        },
        {
          "id": "st-tone",
          "name": "tone",
          "type": "string",
          "value": "tense"
        },
        {
          "id": "st-pacing",
          "name": "pacing",
          "type": "string",
          "value": "fast"
        },
        {
          "id": "st-constraints",
          "name": "constraints",
          "type": "string",
          "value": "keep it under one location"
        },
        {
          "id": "st-bible",
          "name": "storyBibleContext",
          "type": "string",
          "value": "Detective Mara Vance, 38, sharp and distrustful. Partner Eli Cross, 45, charming, hides a gambling debt. Set in 1950s Chicago."
        },
        {
          "id": "st-session",

```

```

        "name": "sessionId",
        "type": "string",
        "value": "manual-test-session"
    }
    ],
    },
    "options": {}
},
"name": "Sample Test Input",
"type": "n8n-nodes-base.set",
"typeVersion": 3.4,
"position": [
    224,
    1456
]
},
{
    "parameters": {
        "mode": "manual",
        "includeOtherFields": false,
        "options": {},
        "assignments": {
            "assignments": [
                {
                    "id": "af-status",
                    "name": "status",
                    "type": "string",
                    "value": "success"
                },
                {
                    "id": "af-action",
                    "name": "action",
                    "type": "string",
                    "value": "full_production_run"
                },
                {
                    "id": "af-session",
                    "name": "session_id",
                    "type": "string",
                    "value": "={{ $(\"Normalize Writer Input\").item.json.sessionId }}"
                },
                {
                    "id": "af-scenes",
                    "name": "scene_suggestions",
                    "type": "string",
                    "value": "={{ $(\"Scene Planner Agent\").item.json.output }}"
                },
                {
                    "id": "af-cont",
                    "name": "continuity_report",
                    "type": "string",
                    "value": "={{ $(\"Continuity Guard Agent\").item.json.output }}"
                },
                {
                    "id": "af-bible",
                    "name": "updated_story_bible",
                    "type": "string",
                    "value": "={{ $json.output }}"
                },
                {
                    "id": "af-report",
                    "name": "full_report",
                    "type": "string",
                    "value": "=# NarrativeAI – Story Production Report\n\n## 1. Scene Suggestions\n{{ $(\"Scene Planner Agent\").item.json.output }}\n\n## 2. Continuity Validation\n{{ $(\"Continuity Guard Agent\").item.json.output }}\n\n## 3. Updated Story Bible\n{{ $json.output }}"
                },
                {
                    "id": "af-msg",
                    "name": "message",
                    "type": "string",
                    "value": "Full narrative production run complete: scenes generated, continuity validated, and story bible updated."
                }
            ]
        }
    },
    "name": "Assemble Final Report",
    "type": "n8n-nodes-base.set",
    "typeVersion": 3.4,
    "position": [
        1024,
        800
    ]
},
{
    "parameters": {
        "mode": "runOnceForAllItems",
        "language": "javaScript",

```



```

"jsCode": "const data = $input.first().json;\n\nfunction buildNarrativePdf(data) {\n  var NL =
String.fromCharCode(10);\n  var BS = String.fromCharCode(92);\n  var SCENES = data.scene_suggestions || '';\n  var
CONT = data.continuity_report || '';\n  var BIBLE = data.updated_story_bible || '';\n  var SESSION = data.session_id
|| '';\n  var pageW = 595.28, pageH = 841.89, margin = 50;\n  var topY = pageH - margin, bottomY = margin + 6;\n\n
function ascii(s) {\n    s = String(s);\n    s =
s.split(String.fromCharCode(0x2014)).join('-').split(String.fromCharCode(0x2013)).join('-');\n    s =
s.split(String.fromCharCode(0x2018)).join('\'').split(String.fromCharCode(0x2019)).join('');\n    s =
s.split(String.fromCharCode(0x201c)).join('\"').split(String.fromCharCode(0x201d)).join('');\n    s =
s.split(String.fromCharCode(0x2022)).join('·').split(String.fromCharCode(0x2026)).join('...');\n    s =
s.split(String.fromCharCode(0x00a0)).join(' ');
    var out = '';\n    for (var i = 0; i < s.length; i++) { var c =
s.charCodeAt(i); if (c >= 32 && c <= 126) out += s.charAt(i); }\n    return out;\n  }\n  function esc(s) {\n    s =
ascii(s); var out = '';\n    for (var i = 0; i < s.length; i++) { var ch = s.charAt(i); if (ch === ' ' || ch === ')
' || ch === BS) out += BS + ch; else out += ch; }\n    return out;\n  }\n  var lines = [];\n  function wrap(text, font,
size, indent) {\n    var usable = (pageW - 2 * margin) - (indent || 0);\n    var maxChars = Math.max(10,
Math.floor(usable / (size * 0.52)));
    var words = ascii(text).split(' ');
    var cur = '';\n    for (var i = 0;
i < words.length; i++) {\n      var w = words[i]; if (!w) continue;\n      if (!cur) cur = w;\n      else if ((cur +
' + w).length <= maxChars) cur += ' ' + w;\n      else { lines.push({ text: cur, font: font, size: size, indent:
indent || 0 }); cur = w; }\n      }\n    }\n    function gap(h) { lines.push({ gap: h }); }\n    function strip(s) { return s.split('***').join(''); }\n
function renderMarkdown(block) {\n    var arr = String(block).split(NL);\n    for (var i = 0; i < arr.length; i++) {\n
      var line = arr[i].replace(/[\ ]+$/, '');
      if (!line) { gap(5); continue; }\n      if (line.indexOf('## ')
=== 0) { gap(4); wrap(strip(line.substring(3)), 'F2', 13, 0); gap(2); continue; }\n      if (line.indexOf('***')
=== 0 && line.lastIndexOf('***') === line.length - 2 && line.indexOf('- ') === -1) { gap(3); wrap(strip(line), 'F2', 11, 0);
continue; }\n      if (line.indexOf('***') === 0) { gap(3); wrap(strip(line), 'F2', 10.5, 0); continue; }\n      if
(line.indexOf('- ') === 0) { wrap('- ' + strip(line.substring(2)), 'F1', 9.5, 12); continue; }\n      wrap(strip(line),
'F1', 9.5, 0);\n      }\n      }\n      lines.push({ text: 'NarrativeAI', font: 'F2', size: 11, indent: 0
});\n      lines.push({ text: 'Story Production Report', font: 'F2', size: 20, indent: 0 });\n      gap(3);\n      lines.push({
text: 'Pipeline: Scene Planner > Continuity Guard > Story Bible', font: 'F1', size: 9, indent: 0 });\n      if (SESSION)
lines.push({ text: 'Session: ' + SESSION, font: 'F1', size: 9, indent: 0 });\n      gap(10);\n      lines.push({ text: '1.
Scene Suggestions', font: 'F2', size: 13, indent: 0 }); gap(4);\n      renderMarkdown(SCENES); gap(8);\n      lines.push({
text: '2. Continuity Validation', font: 'F2', size: 13, indent: 0 }); gap(4);\n      renderMarkdown(CONT); gap(8);\n
      lines.push({ text: '3. Updated Story Bible', font: 'F2', size: 13, indent: 0 }); gap(4);\n      renderMarkdown(BIBLE);\n
      var pages = [], ops = [], y = topY;\n      function flush() { pages.push(ops.join(NL)); ops = []; y = topY; }\n      for
(var i = 0; i < lines.length; i++) {\n        var ln = lines[i];\n        if (ln.gap !== undefined) { y -= ln.gap; if (y <
bottomY) flush(); continue; }\n        var leading = ln.size * 1.45;\n        if (y - leading < bottomY) flush();\n        var x =
(margin + (ln.indent || 0)).toFixed(2);\n        var baseline = (y - ln.size).toFixed(2);\n        ops.push('BT /' + ln.font
+ ' ' + ln.size + ' Tf 1 0 0 1 ' + x + ' ' + baseline + ' Tm (' + esc(ln.text) + ') Tj ET');\n        y -= leading;\n        }\n
      if (ops.length) flush();\n      if (pages.length === 0) pages.push('');\n      var nPages = pages.length;\n      var catalog =
1, pagesObj = 2, fReg = 3, fBold = 4, contentStart = 5, pageStart = 5 + nPages;\n      var kids = [];\n      for (var i = 0; i
< nPages; i++) kids.push((pageStart + i) + ' 0 R');\n      var objects = [];\n      objects[catalog] = '<< /Type /Catalog
/Pages ' + pagesObj + ' 0 R >>';\n      objects[pagesObj] = '<< /Type /Pages /Count ' + nPages + ' /Kids [' + kids.join('
') + ' ] >>';\n      objects[fReg] = '<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica /Encoding /WinAnsiEncoding >>';\n
      objects[fBold] = '<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica-Bold /Encoding /WinAnsiEncoding >>';\n      for
(var i = 0; i < nPages; i++) {\n        var stream = pages[i];\n        var len = Buffer.byteLength(stream, 'latin1');\n
objects[contentStart + i] = '<< /Length ' + len + ' >>' + NL + 'stream' + NL + stream + NL + 'endstream';\n
objects[pageStart + i] = '<< /Type /Page /Parent ' + pagesObj + ' 0 R /MediaBox [0 0 ' + pageW.toFixed(2) + ' ' +
pageH.toFixed(2) + ' ] /Resources << /Font << /F1 ' + fReg + ' 0 R /F2 ' + fBold + ' 0 R >> >> /Contents ' +
(contentStart + i) + ' 0 R >>';\n      }\n      var total = 4 + 2 * nPages;\n      var header = '%PDF-1.4' + NL;\n      var body = '',
offsets = [];\n      for (var n = 1; n <= total; n++) {\n        offsets[n] = header.length + Buffer.byteLength(body,
'latin1');\n        body += n + ' 0 obj' + NL + objects[n] + NL + 'endobj' + NL;\n      }\n      var xrefPos = header.length +
Buffer.byteLength(body, 'latin1');\n      var xref = 'xref' + NL + '0 ' + (total + 1) + NL + '0000000000 65535 f ' + NL;\n
      for (var n = 1; n <= total; n++) xref += String(offsets[n]).padStart(10, '0') + ' 00000 n ' + NL;\n      var trailer =
'trailer' + NL + '<< /Size ' + (total + 1) + ' /Root ' + catalog + ' 0 R >>' + NL + 'startxref' + NL + xrefPos + NL +
'%EOF';\n      return header + body + xref + trailer;\n    }\n\n    const fileName =
'NarrativeAI_Story_Production_Report.pdf';\n    const pdfStr = buildNarrativePdf(data);\n    const buffer =
Buffer.from(pdfStr, 'latin1');\n    const binaryData = await this.helpers.prepareBinaryData(buffer, fileName,
'application/pdf');\n    return [{ json: { status: data.status || 'success', filename: fileName, message: data.message ||
'PDF generated', session_id: data.session_id || '' }, binary: { data: binaryData } }];\n  }\n
},\n  {\n    name: "Generate PDF",\n    type: "n8n-nodes-base.code",\n    typeVersion: 2,\n    position: [\n      1264,\n      800\n    ]\n  },\n  {\n    connections: {\n      "Writer Input Webhook": {\n        main: [\n          {\n            node: "Normalize Writer Input",\n            type: "main",\n            index: 0\n          }\n        ]\n      },\n      "Normalize Writer Input": {\n        main: [\n          {\n            node: "Scene Planner Agent",\n            type: "main",\n            index: 0\n          }\n        ]\n      }\n    }\n  }\n]

```

```

    ]
  },
  "Scene Planner Agent": {
    "main": [
      [
        {
          "node": "Continuity Guard Agent",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Scene Planner LLM": {
    "ai_languageModel": [
      [
        {
          "node": "Scene Planner Agent",
          "type": "ai_languageModel",
          "index": 0
        }
      ]
    ]
  },
  "Story Session Memory": {
    "ai_memory": [
      [
        {
          "node": "Scene Planner Agent",
          "type": "ai_memory",
          "index": 0
        }
      ]
    ]
  },
  "Continuity Guard Agent": {
    "main": [
      [
        {
          "node": "Story Bible Agent",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Continuity Guard LLM": {
    "ai_languageModel": [
      [
        {
          "node": "Continuity Guard Agent",
          "type": "ai_languageModel",
          "index": 0
        }
      ]
    ]
  },
  "Story Bible Agent": {
    "main": [
      [
        {
          "node": "Assemble Final Report",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Story Bible LLM": {
    "ai_languageModel": [
      [
        {
          "node": "Story Bible Agent",
          "type": "ai_languageModel",
          "index": 0
        }
      ]
    ]
  },
  "Run Test (Click Execute)": {
    "main": [
      [
        {
          "node": "Sample Test Input",
          "type": "main",
          "index": 0
        }
      ]
    ]
  }
]

```

```

},
"Sample Test Input": {
  "main": [
    [
      {
        "node": "Normalize Writer Input",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Assemble Final Report": {
  "main": [
    [
      {
        "node": "Generate PDF",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Generate PDF": {
  "main": [
    [
      {
        "node": "Respond with Story Package",
        "type": "main",
        "index": 0
      }
    ]
  ]
}
},
"settings": {
  "executionOrder": "v1",
  "timezone": "Asia/Kolkata"
},
"meta": {
  "aiBuilderAssisted": true
}
}

```